16. Reflection Overview (and When Not to Use It)

16. Reflection Overview (and When Not to Use It)

Phase: Phase 2 — Core language depth
Estimated time: 1 day
Slug: `16-reflection`

Learning objectives

- Explain reflect.Type and reflect.Value at a high level
- Recognize frameworks that use reflection (encoding/json)
- Prefer codegen or generics over heavy reflection
- Measure cost before using reflection in hot paths

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Reflection powers encoding/json and some frameworks — but it is slow and bypasses the type checker. Know it to read libraries; avoid it in app code.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- reflect.TypeOf / ValueOf inspect values at runtime.
- Kind distinguishes struct, slice, pointer, etc.
- Settable values require addressability rules.
- Prefer codegen or generics over reflective maps of fields.
- Interfaces + codegen usually beat reflect for performance.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-reflection && cd /tmp/gw-reflection
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "reflect"
)

func main() {
    var x any = 42
    v := reflect.ValueOf(x)
    fmt.Println(v.Kind(), v.Int())
}
```

Run it:

```
go run .
```

Expected output:

```
int 42
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Print Kind of several values including a struct.
2. Walk exported fields of a struct with reflect.Type.
3. Rewrite a tiny reflective helper as a generic instead.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

What reflection does

The `reflect` package inspects types and values at runtime — powerful and easy to abuse.

```
func KindOf(v any) string {
    return reflect.TypeOf(v).Kind().String()
}
```

`encoding/json` uses reflection to map structs to JSON. That convenience costs CPU and clarity.

Prefer

- Generics for type-safe containers
- Interfaces for behavior
- `go generate` / codegen for repetitive glue

Use reflection for marshaling, debugging tools, and DI frameworks — not business logic.

Failure modes to watch while coding

- Building your whole domain model on map[string]any + reflect
- Ignoring panic risks from invalid reflect usage
- Premature “flexible” APIs that erase types

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- getattr/hasattr are common in Python; in Go they are a smell for app code.
- Duck typing at runtime ≈ interface{} + reflect — prefer interfaces.
- Performance: reflect is convenient and costly.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

If you need reflect, isolate it behind a small package with heavy tests.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Reflection Overview (and When Not to Use It)” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Building your whole domain model on map[string]any + reflect
- Ignoring panic risks from invalid reflect usage
- Premature “flexible” APIs that erase types

Exercises

Work in `exercises/16-reflection/`. Read that folder’s `README.md` before you code.

What you will build

Use reflect.Kind (or the stub's required API) to classify values as tests demand.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/16-reflection/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Name one stdlib package that uses reflection
2. Why are generics often better?
3. What is a hot-path risk of reflect?

Further reading

- https://go.dev/blog/laws-of-reflection
- https://pkg.go.dev/reflect